

ANÁLISIS DEL RETO

Juan Sebastian rojas Marin, 202610140, js.rojasm123@uniandes.edu.co

Katy Terán Mercado, k.teranm@uniandes.edu.co, 202614247

Juan José Rincón Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<6>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

```

410 def req_6(catalog,start_date,end_date):
411     """
412     Retorna el resultado del requerimiento 6
413     """
414     start_time = get_time()
415     orders = catalog["orders"]
416     total= len(orders)
417
418     filter=slt.new_list()
419     for i in range(total):
420         order= lt.get_element(orders,i)
421         date=order["Order_Date"]
422         if start_date <=date <=end_date:
423             slt.add_last(filter, order)
424
425     count=slt.size(filter)
426     channels={}
427     node = filter["first"]
428     while node is not None:
429         order = node["info"]
430         channel=order["Channel"]
431         amount=float(order["Amount"])
432         price=float(order["Price_per_Box"])
433         marketing=float(order["Marketing_Spend"])
434
435         if channel not in channels:
436             channels[channel]={"count":0,
437                                "sum_amount":0,
438                                "sum_price":0,
439                                "sum_marketing":0,
440                                "max_order":None,
441                                "min_order":None
442                             }
443
  
```

```

c=channels[channel]
c["count"]+=1
c["sum_amount"]+=amount
c["sum_price"]+=price
c["sum_marketing"]+=marketing

if c["max_order"] is None or amount>float(c["max_order"]):
    c["max_order"]=order
if c["min_order"] is None or amount<float(c["min_order"]):
    c["min_order"]=order

node = node["next"]

top_channel=None
top_channel_count=0
top_channel_amount=0
top_revenue_channel=None
top_revenue_amount=0
top_revenue_count=0

for name, data in channels.items():
    data["avg_price"] = data["sum_price"] / data["count"] if data["count"] > 0 else 0
    data["avg_marketing"] = data["sum_marketing"] / data["count"] if data["count"] > 0 else 0

    if data["count"] > top_channel_count:
        top_channel_name = name
        top_channel_count = data["count"]
        top_channel_amount = data["sum_amount"]

    if data["sum_amount"] > top_revenue_amount:
        top_revenue_name = name
        top_revenue_amount = data["sum_amount"]
        top_revenue_count = data["count"]
  
```

```

480     end_time = get_time()
481
482     result = {
483         "count": count,
484         "top_channel_name": top_channel_name,
485         "top_channel_count": top_channel_count,
486         "top_channel_amount": top_channel_amount,
487         "top_revenue_name": top_revenue_name,
488         "top_revenue_count": top_revenue_count,
489         "top_revenue_amount": top_revenue_amount,
490         "channels": channels,
491     }
492     return result, delta_time(start_time, end_time)
493
  
```

Este requerimiento recibe el catálogo de pedidos y un rango de fechas (start_date, end_date). En un primer recorrido filtra los pedidos cuya fecha se encuentra dentro del rango solicitado, almacenándolos en una single_linked_list (filter). En un segundo recorrido, realizado directamente sobre los nodos de dicha lista, agrupa los pedidos filtrados por canal (Channel), acumulando para cada canal el conteo de pedidos, la suma del monto, del precio por caja y del gasto en marketing, y conservando el pedido de mayor y de menor monto de cada canal. Finalmente recorre el diccionario de canales para calcular los promedios de cada uno e identificar el canal con mayor número de pedidos y el canal con mayor monto total vendido.

Entrada	catalog,start_date,end_date
Salidas	result, delta_time(start_time, end_time)
Implementado (Sí/No)	Si, implementado por todos los integrantes del grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Obtener la lista de pedidos y su tamaño (lt.size)	$O(1)$
Recorrer todos los pedidos con lt.get_element y filtrar por fecha	$O(n)$
Agregar cada pedido filtrado a la lista enlazada (slt.add_last)	$O(1)$ por pedido
Obtener el tamaño de la lista filtrada (slt.size)	$O(1)$
Recorrer nodo a nodo la lista enlazada filtrada	$O(m)$, m = pedidos filtrados
Actualizar el diccionario de canales (conteo, sumas, pedido máximo/mínimo)	$O(1)$ por nodo
Recorrer el diccionario de canales para calcular promedios e identificar el canal top	$O(c)$ c = canales distintos
TOTAL	$O(n)$

Análisis

La complejidad total del requerimiento es $O(n)$, siendo n el número de pedidos del catálogo. El primer recorrido, para filtrar por fecha, es netamente lineal, y cada adición en la lista enlazada (add_last) es $O(1)$. El segundo recorrido se hace sobre los pedidos ya filtrados, por lo que en el peor caso también es $O(n)$, con actualizaciones del diccionario de canales en $O(1)$. El recorrido final sobre los canales depende de su número, pequeño y constante, así que no afecta el orden.

Se usó una Single Linked List (slt) en vez de un Array List porque no se conoce de antemano cuántos pedidos cumplirán el filtro, la lista se construye dinámicamente con add_last, y luego solo se recorre secuencialmente sin necesitar acceso por posición. Las pruebas confirman un crecimiento lineal consistente con $O(n)$.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.